Formative 2: Multimodal Data Preprocessing

User Identity and Product Recommendation System

Face recognition, voiceprint verification and product recommendation

Team:
AlainPacifique

Arnold Mutara

Desire Gift Mutaganzwa

Ivan Muhoza

African Leadership University
13 November 2026

GitHub repository: github.com/mutaganzwa-desire/multimodal_data_preprocessing

System simulation video: youtu.be/Q9iVa4_k5gY

Tracker Sheet: BSE Group Assignments : Task Sheet_ML-Pipeline_Formative 2 - Dat…

1. Overview

This project builds a system that decides whether a person may see a personalized product recommendation. It follows the User Identity and Product Recommendation System Flow from the assignment: the user is first recognised by their face, the product model then runs for that user, and the prediction is only displayed after the user's voice confirms the transaction. Any failed check leads to an access-denied path.

1.1 System flow

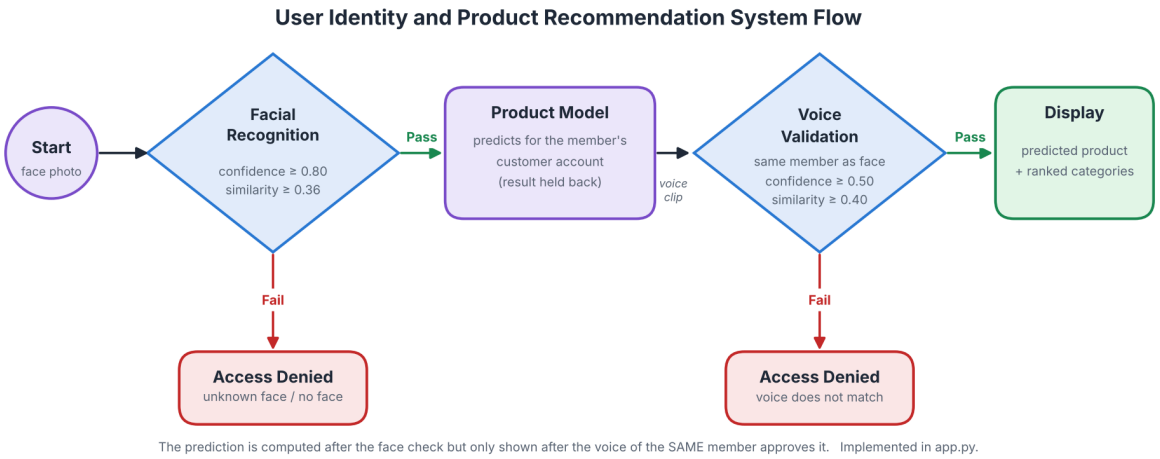


Figure 1. *System flow, following the assignment diagram, as implemented in app.py.*

1. **Face recognition.** A photo is matched against the enrolled team members. An unknown face is denied immediately.
2. **Product prediction.** For the recognised member, the product recommendation model predicts the category their customer account is most likely to buy. The result is held back.
3. **Voiceprint verification.** The member says "Yes, approve" or "Confirm transaction". The voice must belong to the same member the face check recognised. Only then is the prediction displayed; otherwise access is denied.

1.2 What was delivered

Assignment task	Where it is in the repository
1. Data merge	notebooks/Formative_2_Tabular_Data_Preprocessing.ipynb, datasets/merged_customer_recommendation_dataset.csv
2. Image collection and processing	data/images/, src/image_pipeline.py, notebooks/Formative_2_Image_Processing.ipynb, datasets/image_features.csv
3. Sound collection and processing	data/audio/, src/audio_pipeline.py, notebooks/Formative_2_Audio_Processing.ipynb, datasets/audio_features.csv
4. Model creation	src/face_model.py, src/voice_model.py, src/product_model.py, src/verification.py, models/

Assignment task	Where it is in the repository
6. System demonstration	app.py (command-line app), notebooks/Formative_2_Models_and_System_Demo.ipynb

All processing lives in reusable Python modules in src/. The four notebooks call the same functions as the command-line app, so the numbers in the notebooks are exactly the numbers the app uses.

2. Exploratory Data Analysis

2.1 The two datasets

Dataset	Rows	Key columns and types
customer_social_profiles	155 (84 customers)	customer ID (text, e.g. A178), social platform (categorical, 5 values), engagement score (integer 50-99), purchase interest score (float 1.0-5.0), review sentiment (categorical: positive, neutral, negative)
customer_transactions	150 (75 customers)	transaction ID (integer), customer ID (integer), purchase amount (integer), purchase date (text, converted to date), product category (categorical, 5 values: the target), customer rating (float, 10 missing)

Summary statistics, data types, missing values and duplicates were inspected for both tables before any change was made. Transactions run from 2024-01-01 to 2024-05-29, one per day.

2.2 Distributions

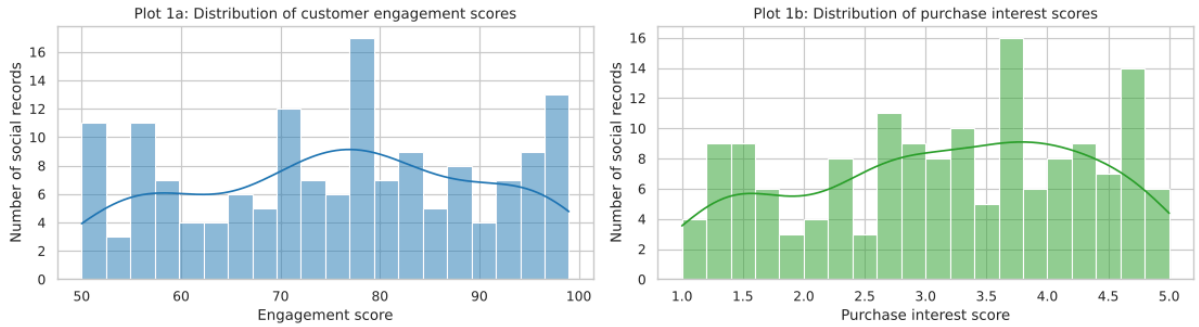


Figure 2. *Distributions of engagement score and purchase interest score in the social profiles.*

Neither score is strongly skewed (skewness -0.11 for engagement and -0.23 for purchase interest), so no transformation is needed. Both cover their full ranges, which means the profiles describe a varied set of customers.

2.3 Outliers

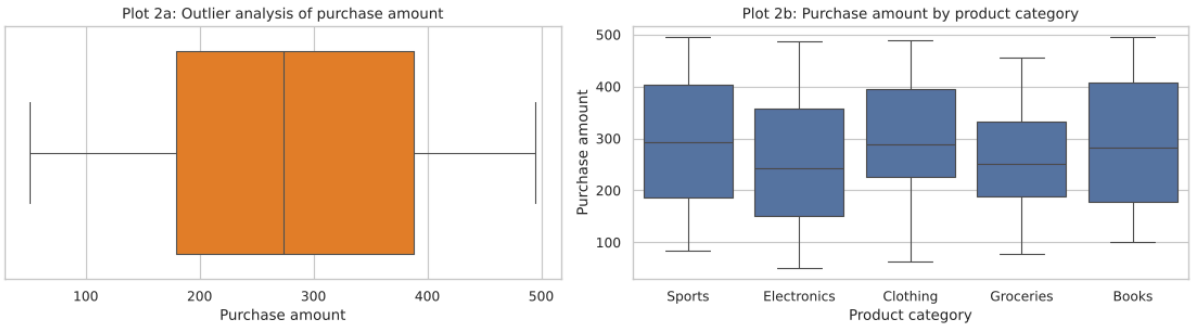


Figure 3. *Purchase amount: overall box plot (outlier check) and distribution per product category.*

The IQR rule (bounds -135 to 703) finds no outliers in the purchase amount. The amounts per category overlap heavily, so the amount alone would not reveal what a customer buys.

2.4 Correlations



Figure 4. *Correlation of the social profile scores and of the transaction values.*

Engagement and purchase interest are almost uncorrelated (r = 0.09), and so are purchase amount and rating (r = 0.03). The two scores therefore carry different information and both are kept as features.

2.5 Target balance

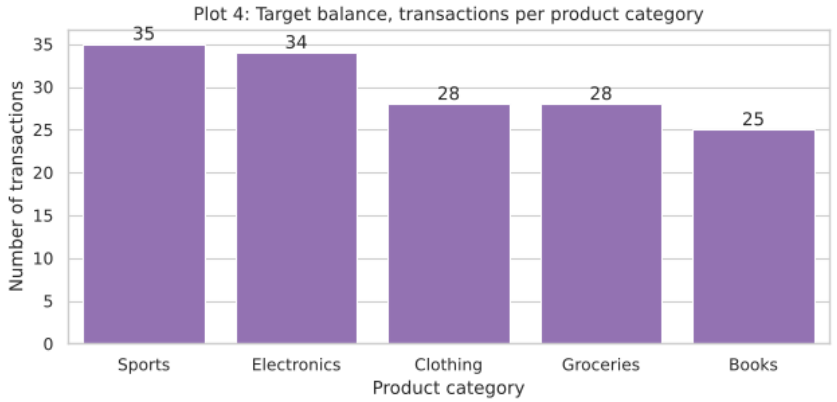


Figure 5. *Number of transactions per product category (the prediction target).*

The five categories have 25 to 35 transactions each. The classes are balanced enough for accuracy to be meaningful, so no resampling was applied; macro F1 is also reported so that every category counts equally.

3. Data Cleaning and Merge Validation

3.1 Cleaning

- **Duplicates:** 5 exact duplicate social records removed.
- **Types and keys:** the two tables wrote the same customer differently (A178 and 178). Both now use an integer customer_id (range 100-199). purchase_date was converted from text to a date.
- **Missing values:** 10 missing ratings (6.7%) were filled with the median (3.0), and a customer_rating_missing flag records which ones were filled.

3.2 Join logic

A customer can have several social records (one per platform), but each transaction must remain exactly one row. Joining the raw tables would duplicate transactions. The social records were therefore first **aggregated to one row per customer**: number of records and platforms, mean and maximum engagement, mean and standard deviation of purchase interest, their product, the share of positive, neutral and negative sentiment, and the dominant platform. This profile was then **left-joined** onto the transactions with pandas validate="many_to_one", which fails loudly if the key is not unique on the profile side.

A left join keeps every transaction. 117 transactions matched a social profile; the 33 transactions from 14 customers without a profile were kept rather than dropped (dropping them would lose 22% of the data). They get has_social_profile = 0, and their profile features are filled with the typical values of profiled customers.

3.3 Post-merge checks

Check	Result
Row count before and after the merge	150 and 150
transaction_id unique after the merge	Yes
Missing values in the merged table	None
Sentiment shares sum to 1 for every row	Yes
Transactions with / without a social profile	117 / 33

Date features (purchase_month, purchase_dayofweek) were added. The purchase amount and rating stay in the file but are not model inputs, because they are only known after the purchase. The result is datasets/merged_customer_recommendation_dataset.csv (150 rows x 21 columns).

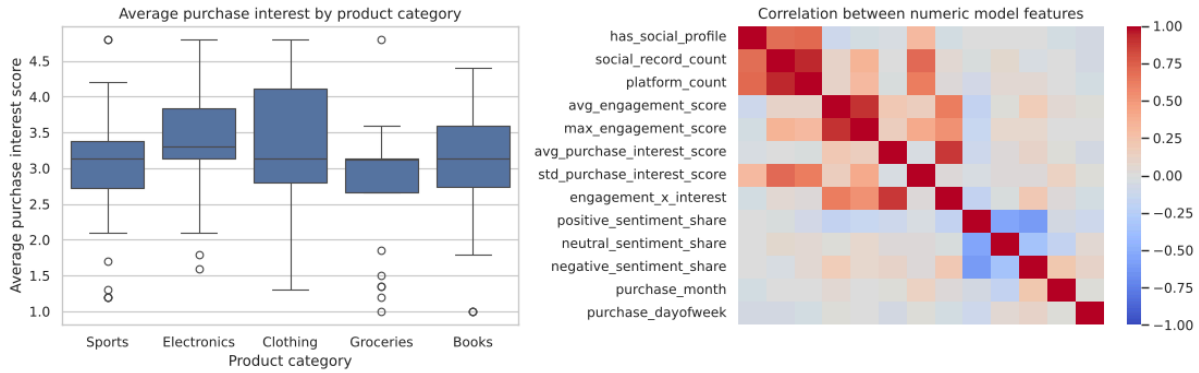


Figure 6. *After the merge: purchase interest per category and correlation between the engineered model features.*

Purchase interest is nearly the same in every category, an early sign that the product category will be hard to predict from these features (confirmed in section 9.4).

4. Image Data Collection

Each of the four members submitted a neutral, a smiling and a surprised photo: 12 photos in total, stored as data/images/<member>/<member>_<expression>.<ext>. The folder name is the identity label and the expression is read from the file name. Arnold's photos were first uploaded outside a member folder; they were moved to data/images/arnold/ and renamed (including fixing a misspelt arnold_nuetral.jpeg), so every file now follows the same pattern.

Resolutions range from 384 x 432 to 2448 x 3264 pixels, taken on different phones, in different rooms and light. This diversity is realistic, but it also means a model could learn the room instead of the person, which is why the face is cropped before any feature is extracted.

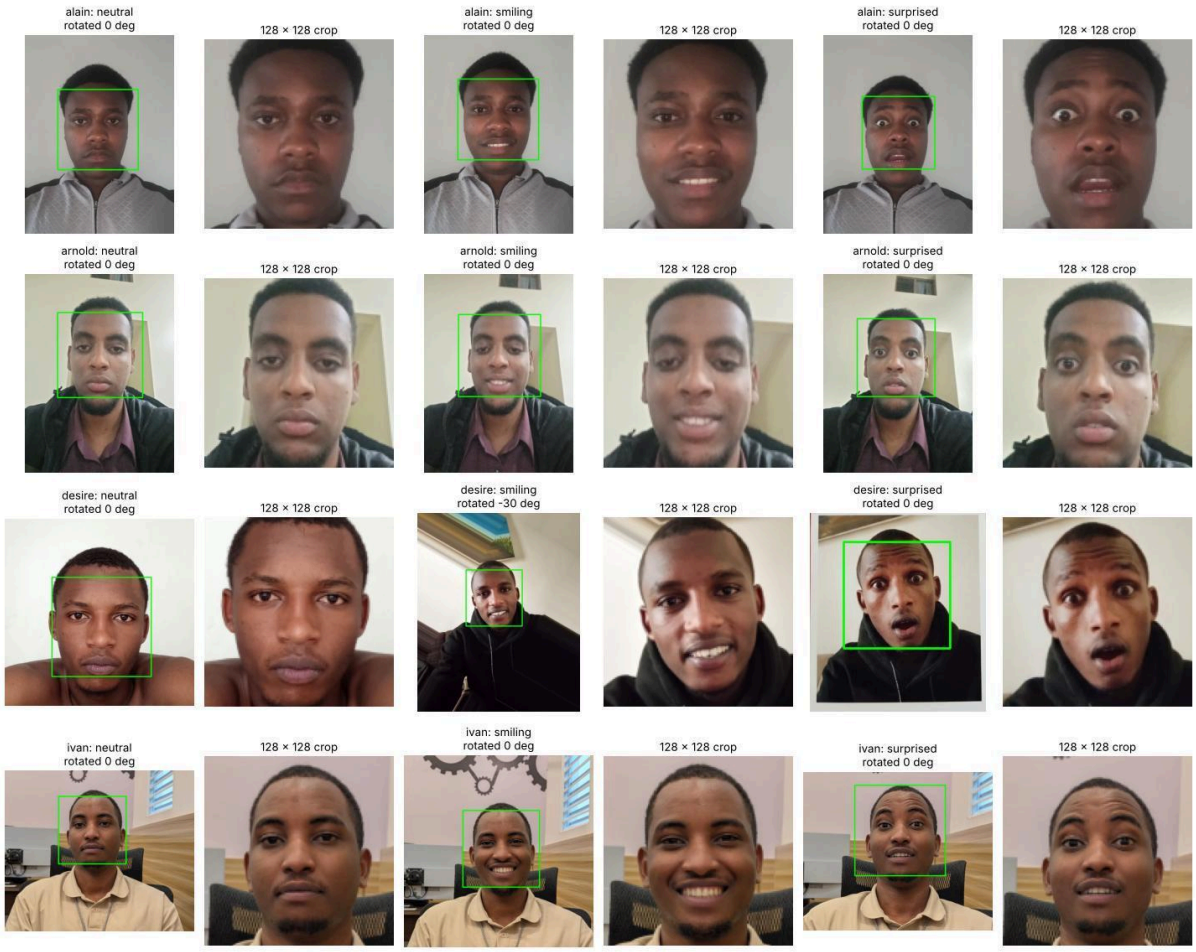


Figure 7. *Every member's three expressions with the detected face (green box) and the 128 x 128 crop used for features.*

A face was found in all 12 photos. In Desire's smiling photo the head is tilted by about 30 degrees; the detector is also run on rotated copies and checks that eyes are present, so the real face is found and straightened.

5. Image Augmentation and Feature Extraction

5.1 Augmentations

Seven augmentations were applied to every face crop, each imitating a real login condition: rotation by +15 and -15 degrees (tilted head), horizontal flip (mirrored selfie camera), grayscale, brighter and darker versions (lighting), and Gaussian noise (a grainy low-light photo). With the original, each photo gives 8 versions.

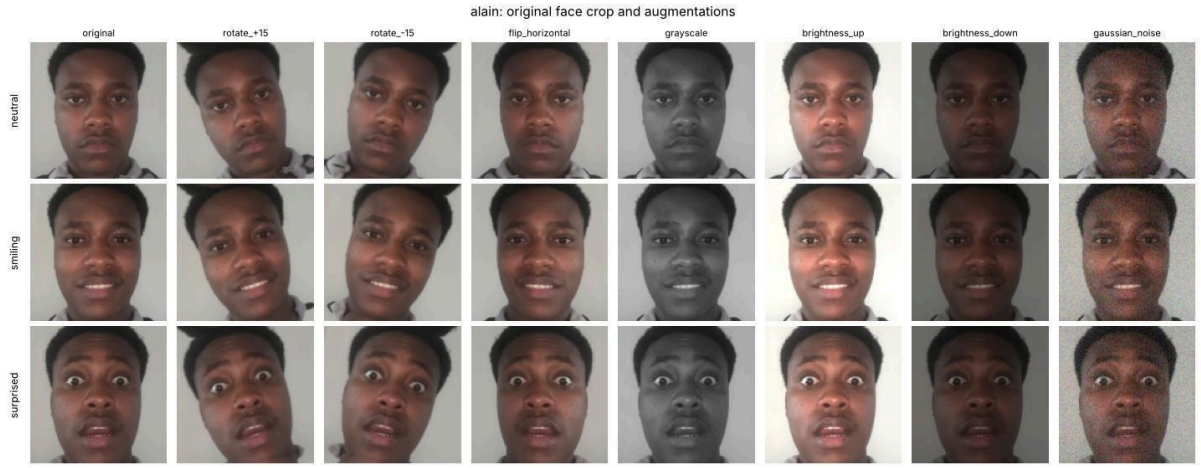


Figure 8. *The seven augmentations applied to one face crop.*

5.2 Features

Each version is described by 532 numbers:

- **Colour histogram (48 values):** 16 bins for each of the red, green and blue channels; captures skin tone and lighting.
- **Local Binary Patterns (160 values):** uniform LBP histograms over a 4 x 4 grid; captures skin and facial texture.
- **Histogram of Oriented Gradients (324 values):** edge directions in 16-pixel cells; captures the shape of the eyes, nose, mouth and jaw.

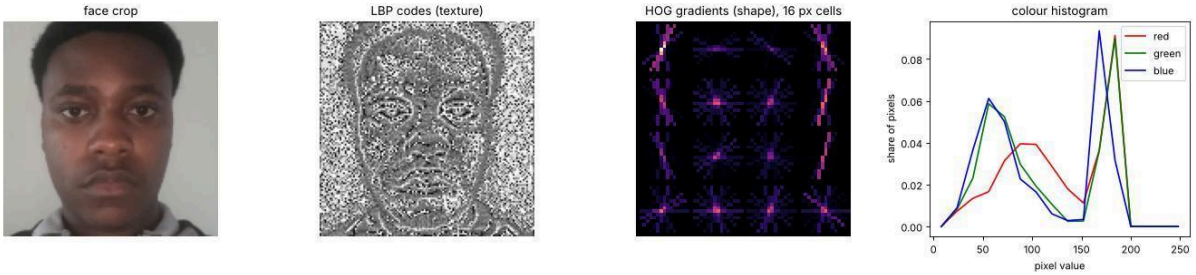


Figure 9. *What each feature block sees: the crop, its LBP texture codes, its HOG gradients and its colour histogram.*

datasets/image_features.csv holds 96 rows (4 members x 3 expressions x 8 versions), 7 metadata columns (member, expression, augmentation, source image and file details) and the 532 feature columns. Validation checks in the notebook confirm there are no missing or infinite values and that every member and expression is present.

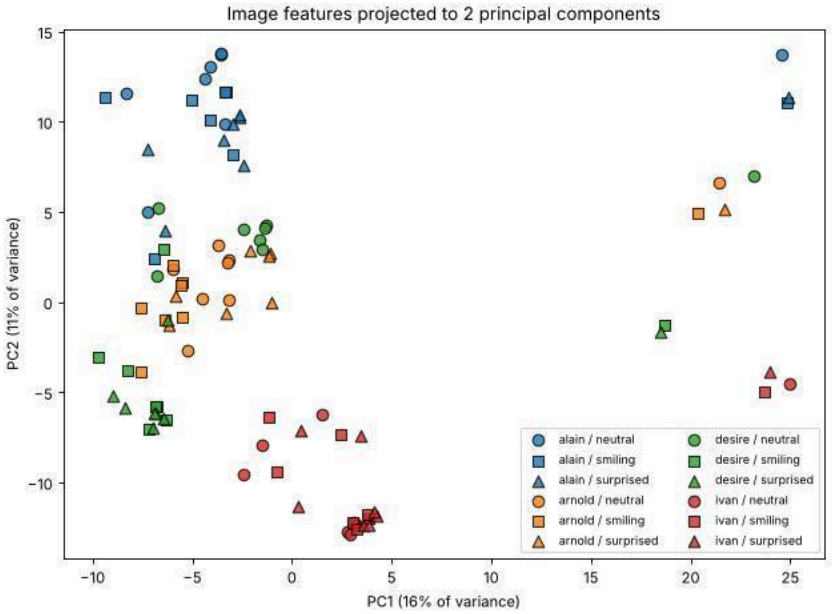


Figure 10. *Image features projected to two principal components, coloured by member.*

Each member's photos form their own region, and a member's three expressions sit close together, which is what face recognition needs. A leave-one-photo-out nearest-neighbour check matches 0.92 of held-out versions to the right member, against 0.25 by chance.

6. Audio Data Collection and Visualisation

Every member recorded both phrases, "Yes, approve" and "Confirm transaction". Alain, Arnold and Desire recorded each phrase twice (4 recordings each). Ivan has one recording per phrase: his second takes were byte-for-byte copies of the first and were removed. In total there are 14 recordings, stored as data/audio/<member>/<member>_<phrase>_<take>.wav.

6.1 Handling different phones and formats

The members used different phones. Ivan's files were phone recordings (MP4/AAC) renamed to .wav, and Arnold's were uploaded outside a member folder. The project now reads files by their content rather than their extension (src/media_io.py): photos in JPEG, PNG, HEIC, WEBP and more, and recordings in WAV, M4A, AAC, MP3, OGG, OPUS and more, with no system software to install. python -m src.media_io converts the data folder to standard WAV and JPEG files with standard names.

6.2 Cleaning

Silence before and after speaking and recording loudness differ by phone. If left in, a model could tell speakers apart by clip length or volume instead of by voice. Each clip is therefore converted to mono 16 kHz and cut to the spoken phrase. A frame counts as speech when it is within 30 dB of the loudest part and at least 12 dB above the recording's own noise floor; the second rule was added for Arnold's first "confirm transaction", where a steady hum only 20 dB below his voice made a fixed rule keep the whole 3.1-second file. Gaps shorter than 0.4 s between words are joined, the most energetic region is kept, and the clip is peak-normalised.

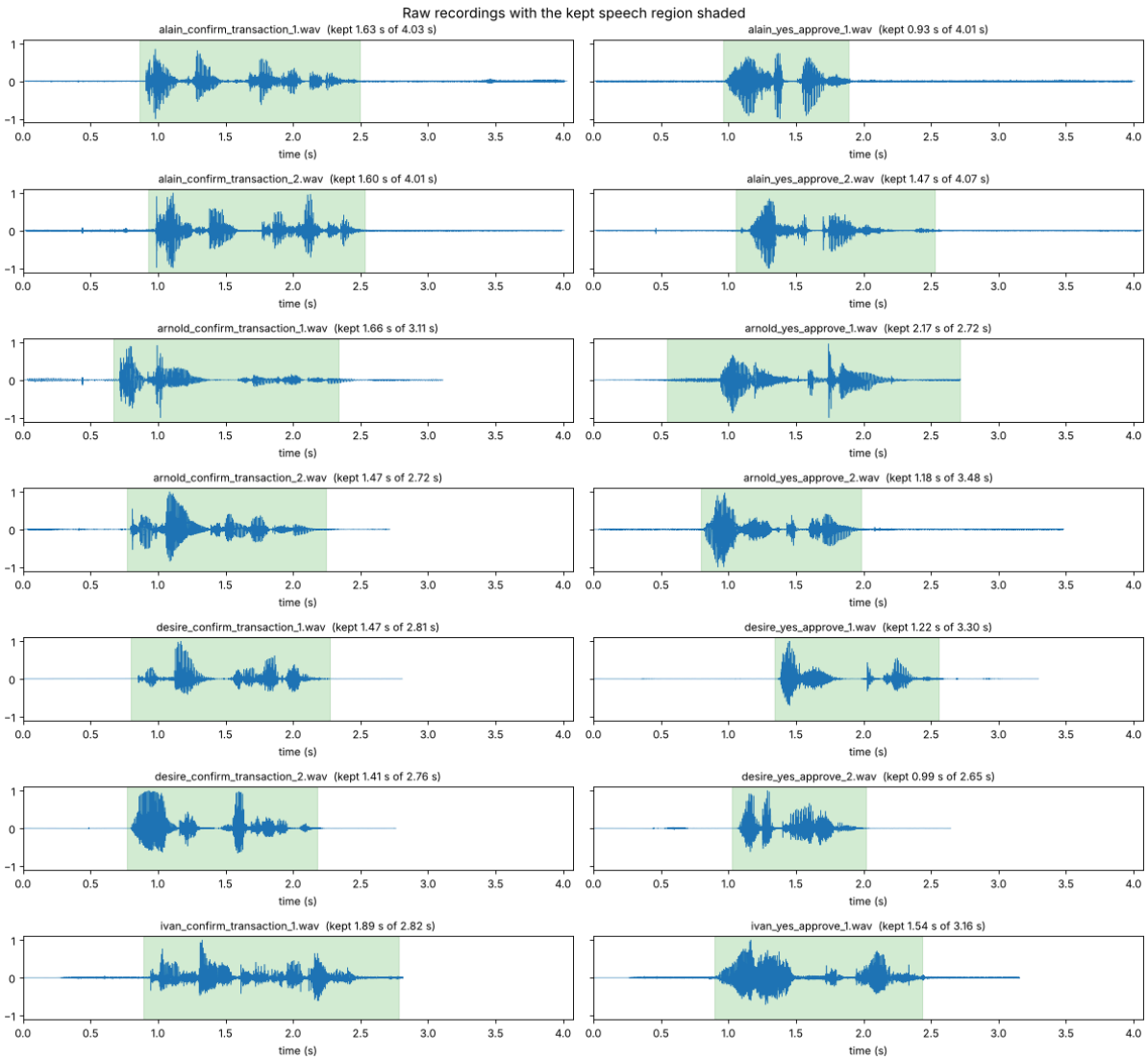


Figure 11. *Raw recordings with the kept speech region shaded green.*

The kept region covers each phrase and excludes silence, a faint sound after Alain's phrases, and the background noise after Ivan's. After cleaning, spoken phrases last 0.9 to 2.2 s.

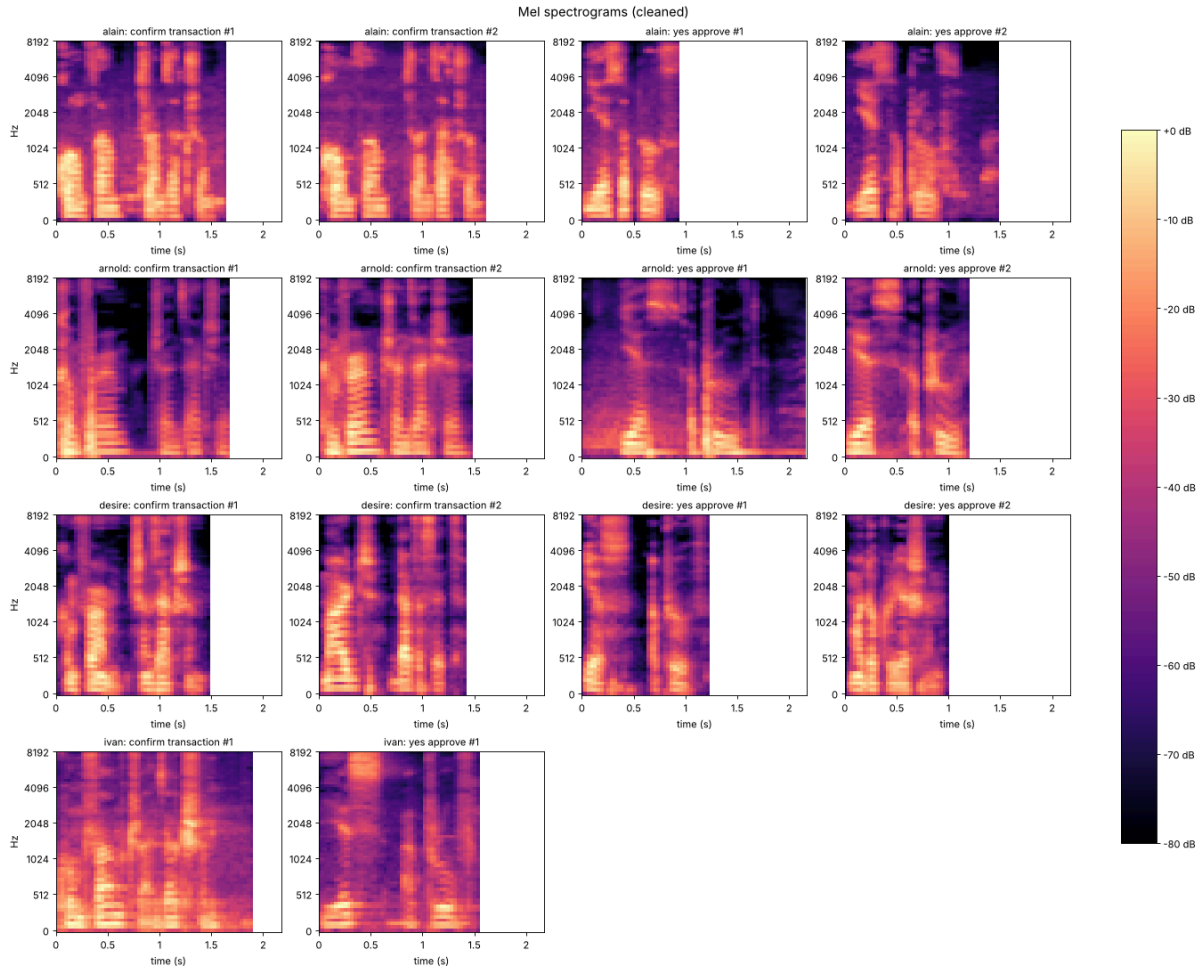


Figure 12. *Mel spectrograms of every cleaned recording.*

Bright horizontal bands below about 1 kHz are the harmonics and formants of the vowels; vertical streaks above 4 kHz are consonants such as "s" and "f". The same phrase shows the same syllable pattern for every speaker (the content), while the spacing and brightness of the low bands differ (the voice), which is what the MFCC features capture. Arnold's energy sits lowest (spectral centroid about 1,670 Hz against about 2,000 Hz for the others). Desire's clips drop to black between words while the others keep a background floor: her phone removed background noise, a device difference noted as a limitation.

7. Audio Augmentation and Feature Extraction

Six augmentations were applied to every recording: pitch shifted up and down by 2 semitones (a different voice pitch on another day), time stretched 15% faster and slower (speaking speed), background noise at 20 dB SNR (a noisy room), and 6 dB quieter (a quieter microphone).

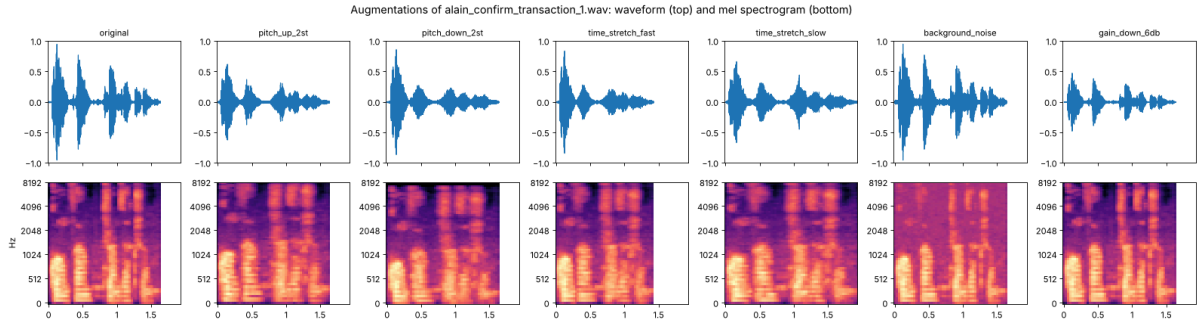


Figure 13. *One recording and its six augmentations as spectrograms.*

Each version is described by 50 features computed over short frames and summarised over time: 13 MFCC means, 13 MFCC standard deviations, 13 standard deviations of the MFCC deltas, spectral roll-off, spectral centroid, spectral bandwidth, RMS energy, zero-crossing rate and duration (with means and standard deviations where relevant). datasets/audio_features.csv holds 98 rows (14 recordings x 7 versions), 6 metadata columns and the 50 feature columns, with no missing values.

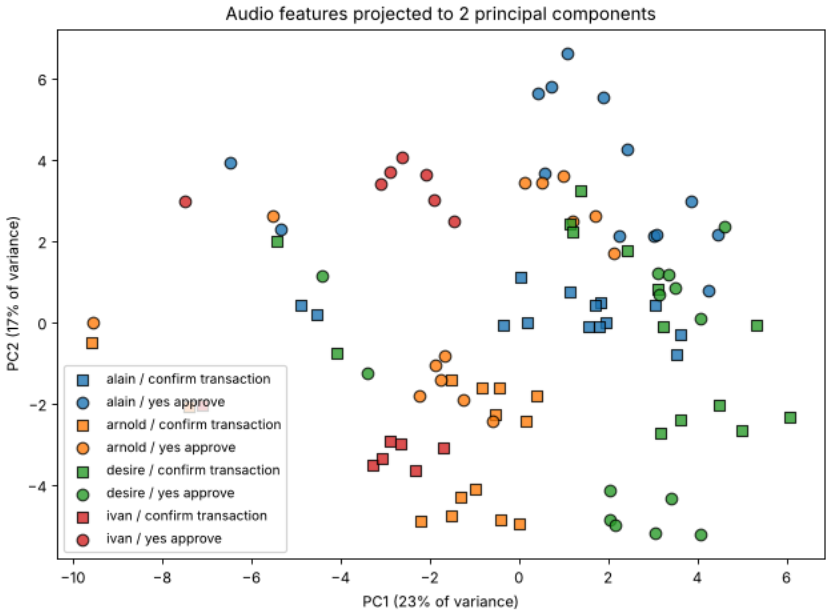


Figure 14. *Audio features projected to two principal components, coloured by speaker and shaped by phrase.*

Desire's and Alain's versions occupy their own regions; Arnold's and Ivan's lie close together, which is the hardest pair to separate. The points furthest left are the background-noise versions: added noise moves the features more than a change of speaker, which is why that augmentation is useful. A leave-one-recording-out nearest-neighbour check matches 0.58 of held-out versions to the right speaker, against 0.25 by chance.

8. Model Implementation

All three models are implemented as Python modules, can be trained from the command line, are saved in models/, and are loaded by the command-line app:

Model	Script	Input	Candidates compared	Selected
Facial recognition	python -m src.face_model	image_features.csv (532 features)	Logistic Regression, Random Forest, XGBoost	Logistic Regression
Voiceprint verification	python -m src.voice_model	audio_features.csv (50 features)	Logistic Regression, Random Forest, XGBoost	Logistic Regression
Product recommendation	python -m src.product_model	merged dataset (14 features)	Baseline, Logistic Regression, Random Forest, XGBoost	Logistic Regression

8.1 Face and voice: identification plus verification

Both biometric models are built by the same procedure (src/verification.py):

1. **Grouped cross-validation.** Models are compared with leave-one-group-out cross-validation, where a group is one source photo or recording. All augmented versions of a sample stay on the same side of the split; a random row split would test the model on near-copies of its training data and inflate the scores.
2. **Model selection by log loss.** The model with the lowest cross-validated log loss is kept, because the decision uses its probabilities, which must therefore be well calibrated.
3. **An outsider check.** A classifier always names one of the members it knows, even for a stranger. So a sample is accepted only if both (a) the classifier's probability for the claimed member reaches a probability threshold and (b) the sample's mean cosine similarity to that member's three closest enrolled samples reaches a similarity threshold.
4. **Thresholds tuned on held-out attempts.** Three kinds of attempt are simulated: genuine members, teammates claiming to be another member, and strangers (each member removed from training in turn and presented as unknown). The thresholds maximise the average of the genuine acceptance rate and the two rejection rates.

8.2 Product recommendation

Inputs are the customer's aggregated social profile (engagement, purchase interest, sentiment shares, number of records and platforms, dominant platform) and the visit date (month and day of week). Purchase amount and rating are excluded because they only exist after the purchase. Evaluation uses 5-fold cross-validation repeated 5 times and **grouped by customer**, so a customer's purchases never appear in both training and test folds. A most-frequent-class baseline shows what "no signal" looks like. Folds are built with NumPy, so the scores are the same on every machine. The model with the best macro F1 is kept.

8.3 Saved models that work on any machine

Each saved model records the scikit-learn version it was built with. If a model file is missing, unreadable or was built with another version, it is retrained automatically from the feature files in a few seconds, so the app never fails because of a different environment. Every notebook's first cell finds or downloads the repository and installs only missing packages; all four notebooks and the app were verified from a fresh clone in an empty Python environment.

9. Evaluation and Multimodal Logic

9.1 Results of all three models

Model	Accuracy	Macro F1	Log loss	Evaluation
Facial recognition (Logistic Regression)	0.917	0.914	0.177	leave one photo out
Voiceprint verification (Logistic Regression)	0.724	0.629	1.002	leave one recording out
Product recommendation (Logistic Regression)	0.243	0.224	1.698	5 x 5-fold CV grouped by customer

9.2 Facial recognition

Model	Accuracy	Macro F1	Log loss
Logistic Regression (selected)	**0.917**	**0.914**	**0.177**
Random Forest	0.906	0.901	0.636
XGBoost	0.802	0.791	0.614

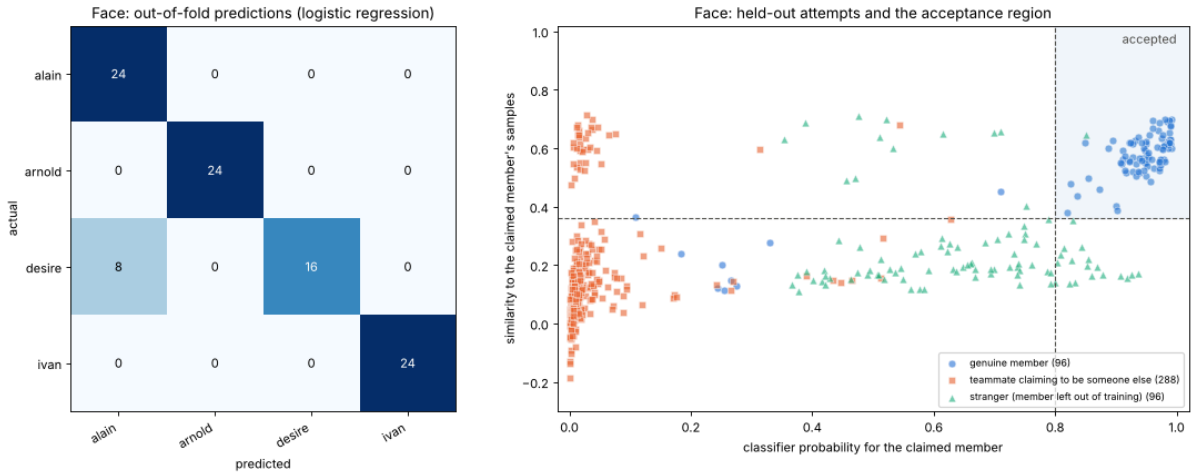


Figure 15. *Face model: out-of-fold confusion matrix (left) and every held-out attempt with the acceptance region (right).*

Logistic Regression has the best accuracy and F1 and by far the lowest log loss: Random Forest is almost as accurate, but its probabilities are much less confident, which would make

a probability threshold unusable. The only errors are the 8 versions of Desire's neutral photo, the one photo taken in a different place and light from her other two.

The scatter plot shows why the second check is needed: strangers often get a classifier probability of 0.4 to 0.9 for some member, but they rarely look similar to that member's enrolled photos. With a probability threshold of 0.80 and a similarity threshold of 0.36, the face check **accepts 91% of genuine attempts (92% of unmodified photos), rejects 100% of teammate impersonations and 99% of strangers**.

9.3 Voiceprint verification

Model	Accuracy	Macro F1	Log loss
Logistic Regression (selected)	**0.724**	**0.629**	**1.002**
Random Forest	0.582	0.482	1.155
XGBoost	0.622	0.536	1.424

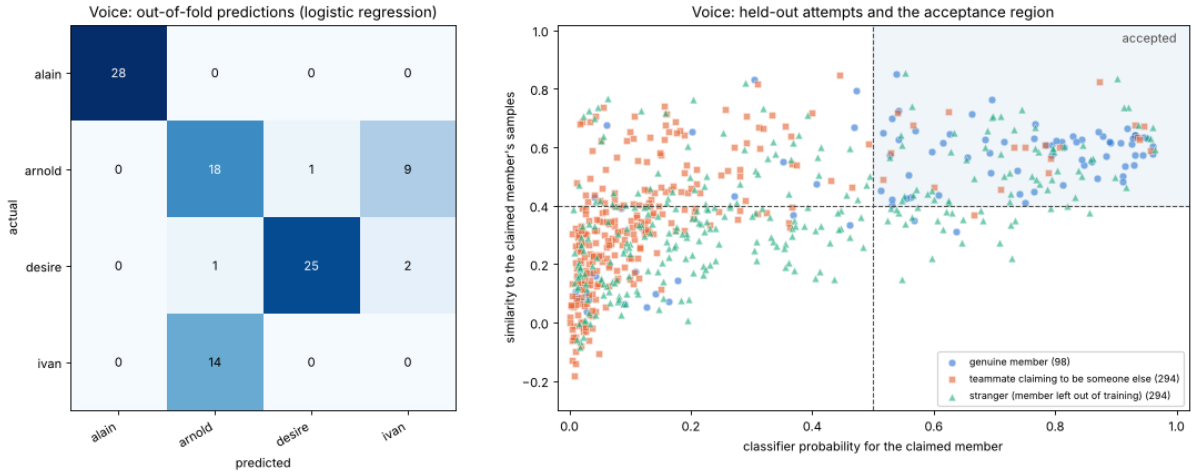


Figure 16. *Voice model: out-of-fold confusion matrix (left) and every held-out attempt with the acceptance region (right).*

Alain is always recognised and Desire in 25 of 28 versions. Arnold is confused with Ivan in 9 of 28 versions, and Ivan is never recognised when one of his recordings is held out: with one recording per phrase, holding one out leaves a single example of his voice, saying the other phrase. Trained on all recordings, as in the app, the model accepts all of Ivan's own recordings (probability about 0.7), but the held-out test shows his voice is not yet reliably separated from Arnold's.

With a probability threshold of 0.50 and a similarity threshold of 0.40, the voice check **rejects 94% of teammate impersonations and 80% of strangers, and accepts 66% of genuine held-out attempts (79% of unmodified recordings)**. The lower genuine rate includes the deliberately distorted versions; a genuine member who is rejected can simply say the phrase again. The voice check is the weaker of the two, and the cause is the amount of audio (2 to 4 recordings per person), not the method.

9.4 Product recommendation

Model	Accuracy	Macro F1	Log loss
Baseline (most frequent category)	0.163	0.088	1.642
Logistic Regression (selected)	**0.243**	**0.224**	1.698
Random Forest	0.219	0.188	1.750
XGBoost	0.195	0.179	1.890

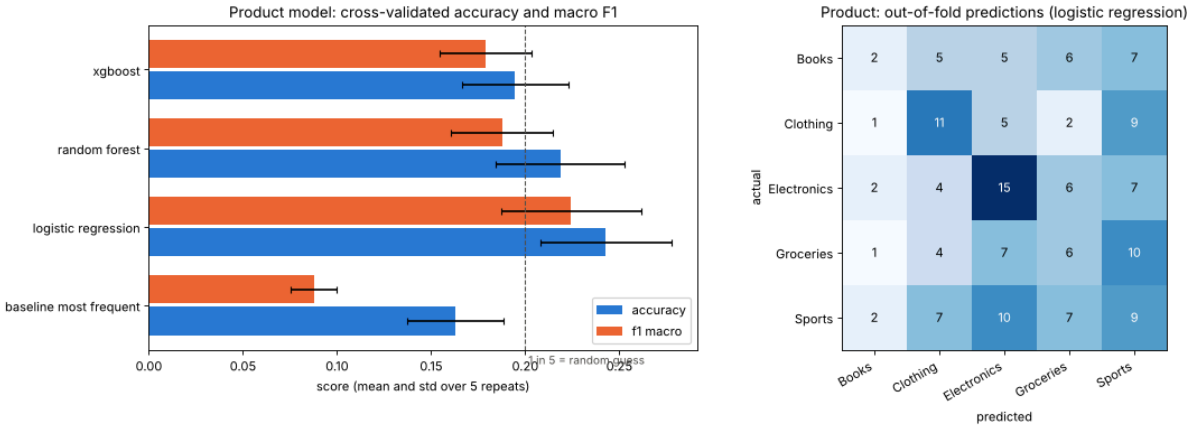


Figure 17. *Product model: cross-validated accuracy and macro F1 of every candidate (left) and out-of-fold confusion matrix (right).*

The product category is close to unpredictable from these features. Logistic Regression beats the baseline on accuracy and clearly on macro F1 (it predicts every category rather than only the most common one), but every model's log loss is above the baseline's, so its probabilities are less reliable than each category's overall share. This matches the EDA: purchase interest and engagement are nearly identical across categories, so the features describe how engaged a customer is, not what they like. The app therefore shows the full ranked list of categories with probabilities rather than one confident answer. Richer behavioural data, such as product views or clicks, would be the main way to improve it.

9.5 Multimodal decision logic

The two biometric modalities guard different steps. The **face** decides who may run a prediction; the **voice** decides whether the prediction may be released. Crucially, the voice is verified against the identity established by the face, not against "any member". This combination is what makes the system secure:

- A stranger is stopped by the face check (99% of held-out stranger photos rejected).
- A teammate holding someone else's photo passes the face check but is stopped by the voice check (94% of teammate voices rejected for the claimed identity).
- A stranger's voice on a genuine member's session is rejected by the voice check (80%).

Neither modality alone could stop both a stranger and a teammate with someone else's photo.

10. System Simulation

The command-line app app.py implements the full flow. It runs interactively (asking for a photo and then a voice recording), for a single transaction with file arguments, or as a scripted demo:

```
python app.py --demo                          # scripted scenarios
python app.py                                 # interactive
python app.py --face photo.jpg --voice clip.m4a    # one transaction
python app.py --face photo.heic --voice clip.wav --customer-id 101 --date 2026-12-01
```

Each member is linked to a customer account in data/member_accounts.csv, which the product model uses. The app exits with code 0 when access is granted and 1 when it is denied, and also denies photos in which no face is found. The scripted demo, also run inside notebook 4, gives:

#	Scenario	Result	Reason shown by the app
1	Alain: own face, own voice	**Granted**	face probability 1.00, similarity 0.93; voice 0.96 / 0.94; recommendation displayed
2	Arnold holds Alain's photo and confirms with his own voice	**Denied at the voice step**	voice probability for Alain 0.014 (sounds most like Arnold)
3	Ketia (not in the team) tries to log in	**Denied at the face step**	best match Desire at 0.518, below 0.80
4	Alain's face but Ketia's voice approves	**Denied at the voice step**	voice probability for Alain 0.368, below 0.50

Scenarios 3 and 4 use a real outsider, Ketia, who is not in the team (data/unauthorized/, never used for training). In addition, when every member in turn plays the stranger, all 12 stranger photos and 34 of 42 stranger voice attempts (81%) are rejected, in line with the held-out rates in section 9.

The demonstration video shows the repository, the scripted demo and interactive runs on the command line, and the models notebook running on Google Colab.

11. Repository and How to Run

Path	Contents
app.py	command-line app (system simulation)
src/media_io.py	reads any image or audio format; python -m src.media_io tidies the data folder
src/image_pipeline.py, src/audio_pipeline.py	face detection, speech trimming, augmentation and feature extraction

Path	Contents
src/verification.py	shared training, evaluation and accept/reject logic for face and voice
src/face_model.py, src/voice_model.py, src/product_model.py	the three models (train with python -m src.<name>)
src/model_store.py, src/environment.py	portable model saving; automatic installation of missing packages
data/images/, data/audio/	members' photos and recordings, one folder per member
datasets/	raw CSVs, merged dataset, image_features.csv, audio_features.csv
models/	trained models
notebooks/	tabular, image, audio, and models + simulation notebooks
reports/	project report

To run everything from scratch on Windows, macOS or Linux:

```
git clone https://github.com/mutaganzwa-desire/multimodal_data_preprocessing.git
cd multimodal_data_preprocessing
pip install -r requirements.txt
python app.py --demo
```

The notebooks can also be opened directly on Google Colab or Kaggle: their first cell downloads the repository and installs missing packages.

12. Limitations and Next Steps

- **Small biometric dataset.** One photo per expression and 2 to 4 recordings per member. The face model's only error and the Arnold/Ivan voice confusion both come from this. More photos in varied places and light, and more takes (in particular a second take from Ivan), would help most.
- **Device bias.** Each member used a different phone, so part of the voice signal may be the device rather than the voice. Recording everyone, including the stranger, on the same phone would remove this.
- **Weak product signal.** The tabular features barely predict the product category; the model is presented as a ranked suggestion. Behavioral data such as views and clicks would improve it.
- **Stranger testing** relies mostly on leaving one member out; only one real outsider (Ketia) was recorded.

13. Team Contributions

See also the team contribution sheet.

Member	Contributions
Desire	Project setup and raw datasets; tabular EDA, cleaning, merge validation and processed dataset; organisation of the members' files; reran the tabular and image notebooks and regenerated their datasets; project report and README; own photos and recordings
Alain	Image and audio preprocessing pipelines and feature files; image and audio notebooks; any-format media loading (src/media_io.py); portable setup, model saving and the shared face/voice verification logic; own photos and recordings
Arnold	Standardised file names and member-account links; unauthorised-samples folder; facial recognition model and product recommendation model; first upload of the models notebook; own photos and recordings
Ivan	Converted his recordings to WAV and regenerated the audio features; voiceprint verification model; command-line app (app.py); models and system demo notebook; own photos and recordings